```
/*!
* @brief
* Base calendar agnostic date. (candidate 2)
*
* @assumptions
* -> Calendar model uses a year, month, and day
* -> Underlying date representation is serial day count
*/
```

struct CalendarDate<Date_Impl, Calendar_Policy, Repr_T, Year_T>

(**Friend**) *Date_Impl*

Nested Types

(**Public**) *Base = CalendarDate<Date_Impl, Calendar_Policy, Repr_T, Year_T>*

(**Public**) *Repr_Type = Repr_T*

(**Public**) *YearInt_t = Year_T*

(**Public**) *CalendarPolicy = Calendar_Policy*

Private Attributes

(**Private**) serialDays: *Repr_Type*

Public Methods

(**Friend**) operator<<(std::ostream& os, const Base& date): return --> *std::ostream&*

(**Public**, **Cexpr**) operator==(const Date_Impl& date): return --> *bool*

(**Public**, **Cexpr**) operator<(const Date_Impl& date): return --> *bool*

(**Public**, **Cexpr**) operator>(const Date_Impl& date): return --> *bool*

(**Public**, **Cexpr**) operator<=(const Date_Impl& date): return --> *bool*

(**Public**, **Cexpr**) operator>=(const Date_Impl& date): return --> *bool*

(**Public**, **Cexpr**) operator+(const Days days): return --> *Date_Impl*

(**Public**, **Cexpr**) operator++(int): return --> *Date_Impl*

(**Public**, **Cexpr**) operator-(const Days days): return --> *Date_Impl*

(**Public**, **Cexpr**) operator-(const Date_Impl date): return --> *Days*

(**Public**, **Cexpr**) operator--(int): return --> *Date_Impl*

(**Public**) operator+=(const Days days): return --> *Date_Impl&*

(**Public**) operator++(): return --> *Date_Impl&*

(**Public**) operator-=(const Days days): return --> *Date_Impl&*

(**Public**) operator--(): return --> *Date_Impl&*

(**Public**, **Cexpr**) isBefore(const Date_Impl& date): return --> *bool*

(**Public**, **Cexpr**) isAfter(const Date_Impl& date): return --> *bool*

(**Public**, **Cexpr**) isBetween(const Date_Impl& start_date, const Date_Impl& end_date): return --> *bool*

(**Public**, **Cexpr**) daysUntil(const Date_Impl& date): return --> *Days*

(**Public**) NAME(): return --> *TYPE*

Private Methods

(**Private**, **Explicit**, **Cexpr**) CalendarDate(const Repr_Type serialDayCount): return --> *void*

(**Private**) ~CalendarDate() = default --> *void*

(**Private**, **Cexpr**) thisImpl(): return --> const *Date_Impl&*

(**Private**) thisImpl(): return --> *Date_Impl&*

(**Private**) NAME(): return --> *TYPE*

```
/*!
* @brief
* Gregorian calendar date.
*/
```

struct GregorianDate : public CalendarDate<GregorianDate, GregorianPolicy, int32_t, gregorian::Year_Type>

Public Attributes

(**Public**, **Static**, **Cexpr**) NAME: *TYPE*

Public Methods

(**Public**, **Cexpr**) Date(const YearInt_t year, const uint8_t month, const uint8_t day): return --> *void*

(**Public**, **Cexpr**) Date(): return --> *void*

(**Public**) ~Date() = default --> *void*

(**Public**, **Cexpr**) isEpoch(): return --> *bool*

(**Public**, **Cexpr**) year(): return --> *YearInt_t*

(**Public**, **Cexpr**) month(): return --> *uint8_t*

(**Public**, **Cexpr**) day(): return --> *uint8_t*

(**Public**) toStr(): return --> *std::string*

(**Public**) NAME(): return --> *TYPE*

Private Methods

(**Public**) NAME(): return --> *TYPE*

```
/*!
* @brief
* Base agnostic calendar system. (candidate 2)
*
* @assumptions
* -> Calendar model uses a year, month, and day
* -> Static API impl. from derivatives
* -> Serial day count based calendar date impl.
*/
```

struct CalendricalSystem<Calendar_Impl, Date_Impl, Month_Enum, DOW_Enum>

(**Friend**) *Calendar_Impl*

Nested Types

(**Public**) *Base = CalendricalSystem<Calendar_Impl, Date_Impl, Month_Enum, DOW_Enum>*

(**Public**) *Date = Date_Impl*

(**Public**) *YearInt_t = typename Date::YearInt_t*

(**Public**) *Month = Month_Enum*

(**Public**) *DayOfWeek = DOW_Enum*

Public Methods

(**Public**, **Static**, **Cexpr**) getMonthName(const uint8_t month): return --> *const char**

(**Public**, **Static**, **Cexpr**) getMonthName(const Month month_repr): return --> *const char**

(**Public**, **Static**, **Cexpr**) getMonthName(const Date date): return --> *const char**

(**Public**, **Static**, **Cexpr**) getMonthAbbrev(const uint8_t month): return --> *std::string_view*

(**Public**, **Static**, **Cexpr**) getMonthAbbrev(const Month month_repr): return --> *std::string_view*

(**Public**, **Static**, **Cexpr**) getMonthAbbrev(const Date date): return --> *std::string_view*

(**Public**, **Static**, **Cexpr**) getMonthEnumRepr(const uint8_t month): return --> *Month*

(**Public**, **Static**, **Cexpr**) getMonthEnumRepr(const Date date): return --> *Month*

(**Public**, **Static**, **Cexpr**) getDayOfWeekName(const DayOfWeek dow_repr): return --> *const char**

(**Public**, **Static**, **Cexpr**) getDayOfWeekName(const Date date): return --> *const char**

(**Public**, **Static**, **Cexpr**) getDayOfWeekAbbrev(const DayOfWeek dow_repr): return --> *std::string_view*

(**Public**, **Static**, **Cexpr**) getDayOfWeekAbbrev(const Date date): return --> *std::string_view*

(**Public**, **Static**, **Cexpr**) getDayOfWeekEnumRepr(const Date date): return --> *DayOfWeek*

(**Public**, **Static**, **Cexpr**) getDate(const YearInt_t year, const uint8_t month, const uint8_t day): return --> *Date*

(**Public**, **Static**, **Cexpr**) getDate(const YearInt_t year, const Month month_repr, const uint8_t day): return --> *Date*

(**Public**, **Static**) getDate(const stl::SystemTimePoint& time_point): return --> *Date*

(**Public**, **Static**) getDate(const stl::SystemTimePoint& time_point, const bool local): return --> *Date*

(**Public**, **Static**, **Cexpr**) getNextDate(const Date from_date, const DayOfWeek dow_repr): return --> *Date*

(**Public**, **Static**, **Cexpr**) getNextDate(const Date from_date, const Month month_repr): return --> *Date*

(**Public**, **Static**, **Cexpr**) getLastDate(const Date from_date, const DayOfWeek dow_repr): return --> *Date*

(**Public**, **Static**, **Cexpr**) getLastDate(const Date from_date, const Month month_repr): return --> *Date*

(**Public**) NAME(): return --> *TYPE*

Private Methods

(**Private**) CalendricalSystem() = delete

(**Private**) ~CalendricalSystem() = delete

```
/*!
* @brief
* Gregorian calendar system.
*/
```

struct GregorianCalendar : public CalendricalSystem<GregorianCalendar, GregorianDate, gregorian::Month, gregorian::DayOfWeek>

Public Attributes

(**Public**, **Static**, **Cexpr**) calendar: *CalendarSystem*

(**Public**, **Static**, **Cexpr**) isSolarCalendar: *bool*

(**Public**, **Static**, **Cexpr**) isLunarCalendar: *bool*

(**Public**, **Static**, **Cexpr**) isLunisolarCalendar: *bool*

(**Public**, **Static**, **Cexpr**) MONTH_NAMES: const *std::array<const char*, gregorian::MONTHS_IN_YEAR>&*

(**Public**, **Static**, **Cexpr**) MONTH_ABBREVS: const *std::array<std::string_view, gregorian::MONTHS_IN_YEAR>&*

(**Public**, **Static**, **Cexpr**) DAY_OF_WEEK_NAMES: const *std::array<const char*, gregorian::DAYS_IN_WEEK>&*

(**Public**, **Static**, **Cexpr**) DAY_OF_WEEK_ABBREVS: const *std::array<std::string_view, gregorian::DAYS_IN_WEEK>&*

Public Methods

(**Public**, **Static**, **Cexpr**) isValidYear(const YearInt_t year): return --> *bool*

(**Public**, **Static**, **Cexpr**) isLeapYear(const YearInt_t year): return --> *bool*

(**Public**, **Static**, **Cexpr**) isLeapYear(const Date date): return --> *bool*

(**Public**, **Static**, **Cexpr**) getDaysInYear(const YearInt_t year): return --> *uint16_t*

(**Public**, **Static**, **Cexpr**) getDaysInYear(const Date date): return --> *uint16_t*

(**Public**, **Static**, **Cexpr**) isValidMonth(const uint8_t month): return --> *bool*

(**Public**, **Static**, **Cexpr**) getDaysInMonth(const YearInt_t year, const uint8_t month): return --> *uint8_t*

(**Public**, **Static**, **Cexpr**) getDaysInMonth(const Date date): return --> *uint8_t*

(**Public**, **Static**, **Cexpr**) isValidDay(const uint8_t day): return --> *bool*

(**Public**, **Static**, **Cexpr**) isValidDOWIndex(const uint8_t dow_index): return --> *bool*

(**Public**, **Static**, **Cexpr**) isValidDate(const YearInt_t year, const uint8_t month, const uint8_t day): return --> *bool*

(**Public**, **Static**, **Cexpr**) isValidDate(const Date date): return --> *bool*

(**Public**, **Static**, **Cexpr**) getDayOfWeekIndex(const YearInt_t year, const uint8_t month, const uint8_t day): return --> *uint8_t*

(**Public**, **Static**, **Cexpr**) getDayOfWeekIndex(const Date date): return --> *uint8_t*

(**Public**, **Static**, **Cexpr**) getWeeksInMonth(const YearInt_t year, const uint8_t month): return --> *uint8_t*

(**Public**, **Static**, **Cexpr**) getWeeksInMonth(const Date date): return --> *uint8_t*

(**Public**, **Static**, **Cexpr**) getWeeksMonthSpans(const YearInt_t year, const uint8_t month): return --> *uint8_t*

(**Public**, **Static**, **Cexpr**) getWeeksMonthSpans(const Date date): return --> *uint8_t*

(**Public**, **Static**, **Cexpr**) toDaysSinceEpoch(const YearInt_t year, const uint8_t month, const uint8_t day): return --> *Days*

(**Public**, **Static**, **Cexpr**) toDaysSinceEpoch(const Date date): return --> *Days*

(**Public**, **Static**, **Cexpr**) fromDaysSinceEpoch(const Days serial_days): return --> *Date*

(**Public**, **Static**, **Cexpr**) toUnixTimestamp(const YearInt_t year, const uint8_t month, const uint8_t day): return --> *stl::UnixTimestamp*

(**Public**, **Static**, **Cexpr**) toUnixTimestamp(const Date date): return --> *stl::UnixTimestamp*

(**Public**, **Static**, **Cexpr**) fromUnixTimestamp(const stl::UnixTimestamp serial_secs): return --> *Date*

(**Public**) NAME(): return --> *TYPE*

Private Methods

(**Private**) GregorianCalendar() = delete

(**Private**) ~GregorianCalendar() = delete